

C++ STL Notes

Everything from the Standard Template Library you need for online tests and coding interviews, explained simply, with code you can trust. Every snippet in these notes was compiled and tested with g++ (C++17).

-
- | | | | |
|-----------|----------------------------------|-----------|------------------------------|
| 1 | Starter template & fast input | 2 | Data types & limits |
| 3 | Reading the constraints | 4 | vector |
| 5 | pair, tuple, structured bindings | 6 | string |
| 7 | stack, queue, deque | 8 | priority_queue (heap) |
| 9 | set & multiset | 10 | map, unordered_map |
| 11 | Algorithms library | 12 | Bits, bitset & maths helpers |
| 13 | Which container should I use? | 14 | Time complexity cheat table |
| 15 | Ready-to-use patterns | 16 | Mistakes that cost marks |
| 17 | STL interview questions | 18 | Before-you-submit checklist |

How to use these notes: read one section a day, then type the code yourself from memory without looking. The goal is that in a test you never have to stop and remember syntax.

1 Starter template & fast input

Type this at the start of every practice problem until you can write it without thinking. On HackerRank function-stub problems, you only fill in the function, so you won't need the `main()` part there.

```
#include <bits/stdc++.h>          // includes the whole standard library (works on g++)
using namespace std;
using ll = long long;           // write ll instead of long long
const int MOD = 1e9 + 7;       // the usual modulus in counting problems

int main() {
    ios::sync_with_stdio(false); // makes cin/cout much faster
    cin.tie(nullptr);

    int t; cin >> t;             // number of test cases
    while (t--) {
        int n; cin >> n;
        vector<int> a(n);
        for (auto &x : a) cin >> x; // read the array
        // ... solve ...
        cout << "answer" << "\n"; // use "\n", not endl (endl is slow)
    }
}
```

Why the two lines in main? By default C++ keeps `cin` in sync with old C input, which is slow. Turning that off can make input 5–10× faster. Once you use them, **don't mix** `cin` with `scanf`/`printf`.

Reading tricky input

```
string line; getline(cin, line); // read a whole line with spaces
cin.ignore();                    // skip the leftover '\n' before getline
while (cin >> x) { ... }        // read until input ends
```

2 Data types & limits

Type	Range (about)	Use it for
<code>int</code>	$\pm 2.1 \times 10^9$	normal counts and indices
<code>long long</code>	$\pm 9.2 \times 10^{18}$	sums, products, anything that can pass 2×10^9
<code>unsigned long long</code>	0 to 1.8×10^{19}	hashing, bit tricks
<code>double</code>	about 15 correct digits	geometry, probability
<code>long double</code>	about 18 correct digits	when double isn't precise enough
<code>char</code>	-128 to 127	letters; <code>c - 'a'</code> gives 0–25

```

INT_MAX, INT_MIN, LLONG_MAX, LLONG_MIN    // built-in limits
ll x = 1LL * a * b;                        // multiply two ints safely: 1LL makes it long long FIRST
ll big = 1e18;                             // fine for initial "infinity" values
cout << fixed << setprecision(6) << ans;  // print a double with 6 decimals

```

Overflow is the #1 reason hidden test cases fail. `int a = 100000, b = 100000; ll c = a * b;` is still wrong, because `a * b` is calculated as `int` first. Write `1LL * a * b`.

3 Reading the constraints

A computer does roughly **10^8 simple operations per second**. So the limit on n tells you how fast your solution has to be, and often hints at the idea the setter wants.

If n is up to	Aim for	Usually means
10–12	$O(n!)$	try all permutations (<code>next_permutation</code>)
20–25	$O(2^n)$	bitmask, backtracking, meet in the middle
100	$O(n^4)$	small DP
500	$O(n^3)$	interval DP, Floyd–Warshall
5 000	$O(n^2)$	2D DP, double loop
$10^5 - 10^6$	$O(n \log n)$ or $O(n)$	sorting, heap, set/map, binary search, two pointers
$10^9 - 10^{18}$	$O(\log n)$ or $O(1)$	binary search on answer, maths, fast power

4 vector

A **vector** is an array that can grow. It's the container you'll use most. Elements sit next to each other in memory, so accessing `v[i]` is instant.

```

vector<int> v;                // empty
vector<int> v(n);             // n zeros
vector<int> v(n, -1);         // n copies of -1
vector<int> v = {4, 1, 3};    // with values

v.push_back(7);              // add at the end          0(1) on average
v.pop_back();                // remove last              0(1)
v.back();                    // last element
v.front();                   // first element
v.size();                    // number of elements (unsigned! see the warning below)
v.empty();                   // true if size is 0
v.clear();                   // remove everything
v.resize(10);                // change size (new slots become 0)
v.assign(n, 0);              // reset to n zeros
v.insert(v.begin() + i, x);   // insert at index i      0(n), slow
v.erase(v.begin() + i);      // remove index i         0(n), slow

for (int x : v) ...           // loop by value (a copy)
for (int &x : v) x *= 2;      // loop by reference (can change elements)

```

2D vectors (grids, DP tables, graphs)

```

vector<vector<int>> grid(n, vector<int>(m, 0)); // n rows, m columns, all 0
grid[i][j] = 5;

vector<vector<int>> adj(n); // graph adjacency list for n nodes
adj[u].push_back(v);      // edge u -> v

```

v.size() is unsigned. If v is empty, v.size() - 1 becomes a huge number, not -1. Write (int)v.size() - 1 or store int n = v.size(); first.

Pass vectors by reference into functions: void dfs(int u, vector<int>& vis). Without the &, C++ copies the whole vector every call, which causes TLE in recursion.

5 pair, tuple, structured bindings

```

pair<int, int> p = {3, 5};
p.first; p.second; // 3, 5
auto p2 = make_pair(1, "x");

vector<pair<int, int>> v; // e.g. (value, index) or (start, end)
sort(v.begin(), v.end()); // sorts by first, then by second

tuple<int, int, int> t = {1, 2, 3};
get<0>(t); // 1

auto [a, b] = p; // structured binding (C++17): a = 3, b = 5
for (auto &[key, val] : mp) ... // very handy with maps

```

Pairs compare **first by .first, then by .second**. That's why sorting a `vector<pair<int,int>>` "just works", and why `{distance, node}` pairs work nicely in a priority queue.

6 string

```
string s = "hello";
s.size(); s.length();           // 5
s[0];                           // 'h'
s += " world";                  // append (fast)
s.push_back('!'); s.pop_back();
s.substr(pos, len);              // part of the string, e.g. s.substr(6, 5) = "world"
s.find("lo");                    // first index of "lo", or string::npos if not found
if (s.find("zz") == string::npos) { /* not found */ }
s.insert(2, "XY"); s.erase(2, 2);
reverse(s.begin(), s.end());
sort(s.begin(), s.end());        // sort the letters (useful for anagrams)

to_string(42);                  // "42"
stoi("42");                     // 42      (string -> int)
stoll("10000000000");           // string -> long long

// character helpers
isdigit(c); isalpha(c); islower(c); isupper(c);
tolower(c); toupper(c);
int idx = c - 'a';               // 'a' -> 0, 'b' -> 1 ...
char ch = 'a' + idx;

// split a line into words
stringstream ss(line); string word;
while (ss >> word) { ... }
```

`s = s + 'x';` inside a loop is $O(n)$ each time because it builds a new string. Use `s += 'x';` or `s.push_back('x');` instead.

7 stack, queue, deque

Container	Think of it as	Main operations (all $O(1)$)	Used for
<code>stack<int></code>	a pile of plates (last in, first out)	<code>push(x)</code> <code>top()</code> <code>pop()</code> <code>empty()</code> <code>size()</code>	brackets, monotonic stack, undo, iterative DFS
<code>queue<int></code>	a line at a counter (first in, first out)	<code>push(x)</code> <code>front()</code> <code>back()</code> <code>pop()</code> <code>empty()</code>	BFS, simulation
<code>deque<int></code>	a line you can join or leave from both ends	<code>push_front</code> <code>push_back</code> <code>pop_front</code> <code>pop_back</code> <code>front()</code> <code>back()</code> <code>dq[i]</code>	sliding window max/min, 0-1 BFS

`pop()` returns nothing. Read the value first with `top()` / `front()`, then pop. Also never call `top()` or `pop()` on an empty stack or queue, as it crashes the program. Check `!st.empty()` first.

8 priority_queue (heap)

A priority queue always gives you the **largest** (or smallest) element quickly. Use it whenever a problem says "top K", "smallest so far", "always pick the cheapest", or for Dijkstra.

```
priority_queue<int> maxHeap; // biggest on top (default)
priority_queue<int, vector<int>, greater<int>> minHeap; // smallest on top

maxHeap.push(5); // O(log n)
maxHeap.top(); // look at the biggest O(1)
maxHeap.pop(); // remove the biggest O(log n)
maxHeap.size(); maxHeap.empty();

// heap of pairs: compares .first, then .second
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq; // min by .first

// custom rule: smallest .second on top
auto cmp = [](const pair<int,int>& a, const pair<int,int>& b) {
    return a.second > b.second; // note: ">" gives a MIN-heap here
};
priority_queue<pair<int,int>, vector<pair<int,int>>, decltype(cmp)> pq2(cmp);
```

The comparator feels "backwards". In sort, $a < b$ means ascending. In a priority_queue, $a < b$ means the **largest** is on top. So to get the smallest on top you write $>$ (or use `greater`). Trick: *multiply values by -1* to turn a max-heap into a min-heap.

Top K largest: keep a **min-heap of size K**. Push each element; if size goes above K, pop. At the end, the heap holds the K largest. Cost: $O(n \log K)$.

9 set & multiset

A **set** keeps values **sorted** and **unique**. A **multiset** keeps them sorted but **allows duplicates**. Both are built on a balanced tree, so most operations are $O(\log n)$.

```

set<int> s;
s.insert(4); s.insert(1); s.insert(4); // s = {1, 4} (duplicate ignored)
s.erase(4); // remove by value
s.count(1); // 1 if present, 0 if not
s.find(1) != s.end(); // another way to check
*s.begin(); // smallest
*s.rbegin(); // largest
s.size();

// the superpower: searching for neighbours
auto it = s.lower_bound(x); // first element >= x (or s.end())
auto it2 = s.upper_bound(x); // first element > x
if (it != s.begin()) { auto prev = std::prev(it); } // largest element < x

multiset<int> ms = {2, 2, 3};
ms.erase(ms.find(2)); // removes ONE copy of 2 -> {2, 3}
ms.erase(2); // removes ALL copies of 2 -> {3}

unordered_set<int> us; // not sorted, O(1) average, no lower_bound

```

Two classic bugs. (1) `ms.erase(x)` deletes **every** copy of `x`; use `ms.erase(ms.find(x))` for one. (2) Always use `s.lower_bound(x)`, **not** `lower_bound(s.begin(), s.end(), x)`. The second form is $O(n)$ on a set and causes TLE.

10 map, unordered_map

A map stores **key** → **value** pairs, with each key appearing once. `map` keeps keys sorted (tree, $O(\log n)$); `unordered_map` doesn't (hash table, $O(1)$ on average).

```

map<string, int> mp;
mp["apple"] = 3; // insert or overwrite
mp["apple"]++; // counting: missing keys start at 0
mp.count("kiwi"); // 0 or 1 (does NOT create the key)
auto it = mp.find("apple"); // iterator, or mp.end() if missing
if (it != mp.end()) cout << it->first << " " << it->second;
mp.erase("apple");
mp.size();

for (auto &[key, val] : mp) cout << key << " " << val << "\n"; // sorted by key

// map-only (sorted) features
mp.begin()->first; // smallest key
mp.rbegin()->first; // largest key
mp.lower_bound(k); // first key >= k

unordered_map<int, int> freq;
for (int x : a) freq[x]++; // frequency count in O(n)
freq.reserve(1 << 20); // optional: fewer rehashes, a bit faster

```

mp[key] creates the key if it's missing. Just reading `if (mp[x] > 0)` adds x with value 0. That can change `mp.size()` and break loops. Use `mp.count(x)` or `mp.find(x)` when you only want to check.

When unordered_map is slow (anti-hash tests)

On Codeforces and some OAs, special inputs make many keys land in the same bucket, so `unordered_map` becomes $O(n)$ per operation and you get TLE. Either switch to `map`, or use this well-known custom hash:

```
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED = chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED);
    }
};
unordered_map<long long, int, custom_hash> safeMap;
```

	map / set	unordered_map / unordered_set
Built using	red-black tree (balanced BST)	hash table
Speed	$O(\log n)$ always	$O(1)$ average, $O(n)$ worst
Keys sorted?	yes	no
lower_bound / smallest / largest	yes	no
Key can be a pair or vector?	yes, works directly	needs a custom hash
Pick it when	you need order or neighbours	you only need fast lookups or counts

11 Algorithms library

These work on any range written as `(begin, end)`. For a vector that's `v.begin(), v.end()`; for a plain array `a` of size `n` it's `a, a + n`.

Sorting

```
sort(v.begin(), v.end()); // ascending O(n log n)
sort(v.begin(), v.end(), greater<int>()); // descending
sort(v.rbegin(), v.rend()); // also descending

// custom rule with a lambda: sort pairs by second value, then first
sort(v.begin(), v.end(), [](const pair<int,int>& a, const pair<int,int>& b) {
    if (a.second != b.second) return a.second < b.second;
    return a.first < b.first;
});
stable_sort(v.begin(), v.end(), cmp); // keeps equal elements in original order
```

Comparator rule: it must return **false** when **a** and **b** are equal. Use `<`, never `<=`. Using `<=` can crash `sort` (it breaks the "strict weak ordering" rule).

Binary search (on a SORTED range)

```
// v = {1, 2, 2, 5, 8}
binary_search(v.begin(), v.end(), 5);           // true / false
lower_bound(v.begin(), v.end(), 2) - v.begin(); // 1  -> first index with value >= 2
upper_bound(v.begin(), v.end(), 2) - v.begin(); // 3  -> first index with value > 2
// how many 2s? upper_bound - lower_bound = 3 - 1 = 2
// if nothing is >= x, lower_bound returns v.end() (index = v.size())
```

Everyday helpers

Code	What it does	Time
<code>reverse(v.begin(), v.end())</code>	reverse in place	O(n)
<code>*max_element(v.begin(), v.end())</code>	largest value (drop * to get the iterator)	O(n)
<code>*min_element(v.begin(), v.end())</code>	smallest value	O(n)
<code>accumulate(v.begin(), v.end(), 0LL)</code>	sum (use 0LL so the sum is long long!)	O(n)
<code>count(v.begin(), v.end(), x)</code>	how many times x appears	O(n)
<code>find(v.begin(), v.end(), x)</code>	iterator to first x, or end()	O(n)
<code>fill(v.begin(), v.end(), 0)</code>	set every element to 0	O(n)
<code>iota(v.begin(), v.end(), 0)</code>	fill with 0, 1, 2, 3 ...	O(n)
<code>v.erase(unique(v.begin(), v.end()), v.end())</code>	remove duplicates (sort first!)	O(n)
<code>next_permutation(v.begin(), v.end())</code>	next arrangement; false after the last one	O(n)
<code>nth_element(v.begin(), v.begin()+k, v.end())</code>	puts the k-th smallest at index k	O(n) avg
<code>partial_sum(a.begin(), a.end(), p.begin())</code>	prefix sums	O(n)
<code>rotate(v.begin(), v.begin()+k, v.end())</code>	rotate left by k	O(n)
<code>swap(a, b)</code>	swap two values or containers	O(1)
<code>max({a, b, c})</code>	max of many values	O(k)

```
// all permutations of {1, 2, 3}: start from the SORTED order
sort(p.begin(), p.end());
do { /* use p */ } while (next_permutation(p.begin(), p.end()));
```

12 Bits, bitset & maths helpers

```
__builtin_popcount(x);      // number of 1 bits in an int    (popcountll for long long)
__builtin_ctz(x);           // trailing zeros (x must not be 0)
__builtin_clz(x);           // leading zeros  (x must not be 0)
x & 1                       // is x odd?
x & (1 << i)                // is bit i set?
x | (1 << i)                // set bit i
x & ~(1 << i)               // clear bit i
x & (x - 1)                 // remove the lowest set bit; ==0 means x is a power of 2
1LL << 40                   // shifting past 31? use 1LL, or it overflows

for (int mask = 0; mask < (1 << n); mask++)      // all subsets of n items
    for (int i = 0; i < n; i++)
        if (mask & (1 << i)) { /* item i is in this subset */ }

bitset<8> b(5);          // 00000101
b.count(); b[0]; b.set(3); b.reset(0); b.to_string();

gcd(a, b); lcm(a, b);    // C++17, in <numeric>
abs(x); sqrtl(x); pow(a, b) // pow uses doubles: DON'T use it for exact integer powers
```

Rounding up division: $(a + b - 1) / b$ for positive a, b . **Modulo of a negative number:** $((x \% \text{MOD}) + \text{MOD}) \% \text{MOD}$, because in C++ $-7 \% 3$ is -1 , not 2 .

13 Which container should I use?

If the problem needs...	Use
a list of items, access by index	<code>vector</code>
count how often each value appears	<code>unordered_map<int,int></code> (or <code>vector</code> if values are small)
check "have I seen this?"	<code>unordered_set</code>
values kept sorted, with insert/delete	<code>set</code> / <code>multiset</code>
"smallest value $\geq x$ " or nearest neighbour	<code>set::lower_bound</code> / <code>map::lower_bound</code>
always grab the max or min quickly	<code>priority_queue</code>
running median	two heaps (max-heap + min-heap) or <code>multiset</code>
last-in-first-out (matching brackets, next greater)	<code>stack</code>
level-by-level search (BFS)	<code>queue</code>
add/remove at both ends; sliding window max	<code>deque</code>
key $\rightarrow$ value lookups	<code>unordered_map</code> (fast) or <code>map</code> (sorted)
a pair or vector as a key	<code>map<pair<int,int>,int></code> / <code>set<vector<int>></code>
flags for many small items, bitmask speed-ups	<code>bitset</code> / <code>vector<bool></code>

14 Time complexity cheat table

Container	Access	Search	Insert	Delete	Notes
vector	$O(1)$	$O(n)$	$O(1)$ * end / $O(n)$ middle	$O(1)$ end / $O(n)$ middle	*amortised (average)
deque	$O(1)$	$O(n)$	$O(1)$ both ends	$O(1)$ both ends	
stack / queue	top/front $O(1)$	–	$O(1)$	$O(1)$	
priority_queue	top $O(1)$	–	$O(\log n)$	$O(\log n)$ top only	no search, no delete-any
set / map	–	$O(\log n)$	$O(\log n)$	$O(\log n)$	sorted, lower_bound
multiset	–	$O(\log n)$	$O(\log n)$	$O(\log n)$	count(x) is $O(\log n + k)$
unordered_set / map	–	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	$O(n)$ worst case
sort					$O(n \log n)$
lower_bound on vector		$O(\log n)$			range must be sorted

15 Ready-to-use patterns

These are the snippets that come up again and again in OAs. Each one is tested.

Prefix sums + hash map: count subarrays with sum = k

```
int subarraySumK(vector<int>& a, int k) {
    unordered_map<long long, int> seen;
    seen[0] = 1; // empty prefix
    long long sum = 0; int ans = 0;
    for (int x : a) {
        sum += x;
        if (seen.count(sum - k)) ans += seen[sum - k];
        seen[sum]++;
    }
    return ans;
}
```

Sliding window maximum (deque)

```
vector<int> slidingMax(vector<int>& a, int k) {
    deque<int> dq;                // stores INDICES; their values decrease front -> back
    vector<int> res;
    for (int i = 0; i < (int)a.size(); i++) {
        while (!dq.empty() && dq.front() <= i - k) dq.pop_front();    // out of window
        while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();    // useless now
        dq.push_back(i);
        if (i >= k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}
```

Next greater element (monotonic stack)

```
vector<int> nextGreater(vector<int>& a) {
    int n = a.size();
    vector<int> ans(n, -1);
    stack<int> st;                // indices still waiting for a bigger value
    for (int i = 0; i < n; i++) {
        while (!st.empty() && a[st.top()] < a[i]) { ans[st.top()] = a[i]; st.pop(); }
        st.push(i);
    }
    return ans;
}
```

Merge overlapping intervals

```
vector<vector<int>> mergeIntervals(vector<vector<int>>> v) {
    sort(v.begin(), v.end());    // by start time
    vector<vector<int>>> res;
    for (auto &x : v) {
        if (res.empty() || res.back()[1] < x[0]) res.push_back(x);    // no overlap
        else res.back()[1] = max(res.back()[1], x[1]);                // extend
    }
    return res;
}
```

Longest increasing subsequence in $O(n \log n)$

```
int lis(vector<int>& a) {
    vector<int> tails;           // tails[i] = smallest tail of an increasing run of length i+1
    for (int x : a) {
        auto it = lower_bound(tails.begin(), tails.end(), x); // use upper_bound for non-
decreasing
        if (it == tails.end()) tails.push_back(x);
        else *it = x;
    }
    return tails.size();
}
```

Coordinate compression (big values $\rightarrow$ 0, 1, 2 ...)

```
vector<int> s = a;
sort(s.begin(), s.end());
s.erase(unique(s.begin(), s.end()), s.end());
for (int &x : a) x = lower_bound(s.begin(), s.end(), x) - s.begin();
// {100, 5, 100, 7}  $\rightarrow$  {2, 0, 2, 1}
```

Binary search on the answer

```
// find the SMALLEST x in [lo, hi] for which ok(x) is true (ok must be false...false true...true)
long long lo = 0, hi = 1e18;
while (lo < hi) {
    long long mid = lo + (hi - lo) / 2; // avoids overflow
    if (ok(mid)) hi = mid;
    else lo = mid + 1;
}
// answer = lo
```

BFS on a grid

```
int dr[] = {-1, 1, 0, 0}, dc[] = {0, 0, -1, 1};
vector<vector<int>> dist(n, vector<int>(m, -1));
queue<pair<int,int>> q;
q.push({sr, sc}); dist[sr][sc] = 0;
while (!q.empty()) {
    auto [r, c] = q.front(); q.pop();
    for (int d = 0; d < 4; d++) {
        int nr = r + dr[d], nc = c + dc[d];
        if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue; // outside grid
        if (grid[nr][nc] == '#' || dist[nr][nc] != -1) continue; // wall or visited
        dist[nr][nc] = dist[r][c] + 1;
        q.push({nr, nc});
    }
}
```

Dijkstra (shortest paths, non-negative weights)

```
// adj[u] = list of {v, w}
vector<long long> dijkstra(int src, vector<vector<pair<int,int>>>& adj) {
    int n = adj.size();
    vector<long long> dist(n, LLONG_MAX);
    priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq;
    dist[src] = 0; pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;           // old entry, skip it
        for (auto [v, w] : adj[u])
            if (dist[u] + w < dist[v]) { dist[v] = dist[u] + w; pq.push({dist[v], v}); }
    }
    return dist;
}
```

Disjoint Set Union (union-find)

```
struct DSU {
    vector<int> p, sz;
    DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); } // path compression
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;           // already in the same group
        if (sz[a] < sz[b]) swap(a, b);      // union by size
        p[b] = a; sz[a] += sz[b];
        return true;
    }
};
```

Fast power ($a^b \bmod m$) and the prime sieve

```
long long power(long long b, long long e, long long m = MOD) {
    long long r = 1; b %= m;
    while (e > 0) {
        if (e & 1) r = r * b % m;
        b = b * b % m;
        e >>= 1;
    }
    return r;
}

// modular inverse when MOD is prime: inv(a) = power(a, MOD - 2)

vector<int> sieve(int n) { // all primes <= n in O(n log log n)
    vector<bool> isPrime(n + 1, true);
    isPrime[0] = false; if (n >= 1) isPrime[1] = false;
    for (int i = 2; 1LL * i * i <= n; i++)
        if (isPrime[i])
            for (int j = i * i; j <= n; j += i) isPrime[j] = false;
    vector<int> primes;
    for (int i = 2; i <= n; i++) if (isPrime[i]) primes.push_back(i);
    return primes;
}
```

16 Mistakes that cost marks

1. **Overflow.** Sums and products in `int`. Use `long long` and `1LL * a * b`.
2. `accumulate(..., 0)` sums in `int`. Write `0LL`.
3. `v.size() - 1` on an empty vector becomes a huge number (unsigned).
4. `mp[x]` to check a key silently inserts it. Use `count` / `find`.
5. `multiset.erase(x)` removes all copies. Use `erase(find(x))`.
6. `lower_bound(s.begin(), s.end(), x)` on a set is $O(n)$. Use `s.lower_bound(x)`.
7. Comparator with `<=` can crash `sort`. Use `<`.
8. Forgetting the `priority_queue` is a max-heap by default.
9. Calling `top()` / `front()` / `pop()` on an empty container.
10. Erasing from a container while looping over it with a range-for. Use the iterator returned by `erase`.
11. Passing vectors by value in recursion → TLE. Use `&`.
12. `endl` in big outputs → slow. Use `"\n"`.
13. `pow()` for integer powers gives rounding errors. Write a loop or fast power.
14. Not resetting global arrays/containers between test cases.
15. Negative modulo: $((x \% MOD) + MOD) \% MOD$.
16. `1 << 40` overflows. Write `1LL << 40`.
17. `unordered_map` TLE on anti-hash tests. Switch to `map` or a custom hash.
18. Deep recursion ($n \approx 10^5$ – 10^6) can overflow the stack. Use an explicit stack or BFS.

17 STL interview questions

Interviewers like to check that you know what's *inside* the tools you use. Short answers you can give confidently:

Q. map vs unordered_map?

map is a red-black tree: keys sorted, $O(\log n)$ per operation. unordered_map is a hash table: no order, $O(1)$ average but $O(n)$ worst case when many keys collide.

Q. How does vector grow?

When it's full it allocates a bigger block (usually $1.5\times$ to $2\times$ the size), copies or moves the elements across, and frees the old block. That's why push_back is $O(1)$ amortised: the occasional expensive copy is spread over many cheap pushes.

Q. size vs capacity? What does reserve do?

size = elements stored; capacity = space already allocated. reserve(n) allocates space up front so no regrowth happens while you push up to n items.

Q. vector vs list?

vector is one continuous block: fast indexing, cache-friendly. list is a doubly linked list: $O(1)$ insert/erase at a known position, but no indexing and poor cache use. In practice vector is usually faster.

Q. What algorithm does std::sort use?

Introsort: quicksort, switching to heapsort if recursion gets too deep, and insertion sort for small parts. Guaranteed $O(n \log n)$. It's not stable; use stable_sort if order of equal items matters.

Q. What is iterator invalidation?

After some changes, old iterators/pointers no longer point to valid elements. Example: push_back that causes a vector to regrow invalidates all its iterators. Erasing from a map invalidates only the erased element's iterator.

Q. How is priority_queue built?

It's a binary heap stored inside a vector. push and pop are $O(\log n)$, top is $O(1)$.

Q. What is a hash collision and how is it handled?

Two keys hash to the same bucket. The standard unordered_map keeps a list per bucket (chaining) and rehashes into more buckets when the load factor gets too high.

Q. Why can't I use a pair as an unordered_map key directly?

There's no built-in std::hash for pair. Either use map (which only needs $<$), or write your own hash.

Q. emplace_back vs push_back?

push_back takes an already-built object; emplace_back builds the object in place from its constructor arguments, which can avoid a temporary copy.

Q. What does stable mean for a sort?

Elements that compare equal keep their original relative order.

Q. What are lower_bound and upper_bound?

On a sorted range: lower_bound gives the first element $\geq x$, upper_bound gives the first element $> x$. Both are $O(\log n)$.

18 Before-you-submit checklist

- ☐ Did I read the constraints and is my complexity fast enough?
- ☐ Could any value overflow int? (sums, products, counts)
- ☐ Did I test $n = 1$, $n = 0$ (if allowed), all elements equal?
- ☐ Did I test the largest values?
- ☐ Negative numbers handled? Negative modulo fixed?
- ☐ Containers and globals reset between test cases?
- ☐ No debug prints left in the output?
- ☐ Output format exactly right? (spaces, new lines, case)
- ☐ On a function-stub problem: no extra `main()` or `cin`?
- ☐ Stuck for 20 minutes? Submit the brute force for partial marks.

Good luck with your placements. Practise typing these until they feel automatic.